Jump → address → label → code segment

→ the most vulnerable instruction of Assembly.

conditional jumps

cmp a,b	unsigned (a, b)	signed (g, l)
a = b	je	je
a ≠ b	jne	jne
a > b	ja , jnbe	jg , jnle
a < b	jb , jnae	jl , jnge
a ≥ b	jae , jnb	jge , jnl
a ≤ b	jbe , jna	jle , jng

cmp a,b → they change FLAGS

→ virtual subtraction (sub a,b) but with no change except set flags → CF, OF, SF, ZF.

→ How do those FLAGS change?

a == b → ZF (a-b = 0)

a < b → CF (a-b < 0)

→ OF

→ PF

→ AF

↓

CF ~ unsigned

```
mov ax,5
cmp ax,5
```
↓ 5-5 = 0 ↓ ZF = 1.

```
mov al,2
cmp al,5
```
↓ 2-5 = -3 ↓ CF = 1 (borrowed)

```
mov al,100
cmp al,-100
```
↓ 100 - (-100) = 200 ↓ OF = 1

```
mov al,8
cmp al,1   → 8-1 = 7 = 00000111 → PF = 0
cmp al,2   → 8-2 = 6 = 00000110 → PF = 1.
```
(3 ones / 2 ones)

SF ~ signed

```
mov al,10h
cmp al,1
```
↓ 0001 0000

```
mov al,2
cmp al,5
```
↓ 2-5 = -3 ↓

0000 1111

↓

AF = 1

SF = 1

Unsigned → CF → cmp al,2 / jb

Signed → SF + OF → cmp al,2 / jl

jc (CF=1)	jp (PF=1)	jz (ZF=1)
jnc (CF=0)	jnp (PF=0)	jnz (ZF=0)

...

```
a - byte
if a>0 then a--
endif
```

→

```
start:
    cmp byte[a], 0
    jbe end_if
        dec byte[a]
    end_if:
```

```
a,b - bytes, signed
if a>10 then
    a--
else
    b--
endif
```

→

```
start:
    cmp byte[a], 10
    jbe else
        dec byte[a]
        jmp end_if
    else:
        dec byte[b]
    end_if:
```

Loops:

```
mov ecx, 5 ; 0? → if we put 0 → inf loop iterations
jecxz end_loop
my_repeat:
    inc byte[a]
loop my_repeat
end_loop:
```

`loop my_repeat` { `dec ecx` / `jecxnz my_repeat`

esi → index for source → i

edi → index for dest → j

→ 2 bytes.

1. Compute the sum of ... of an array of words.

```
bits 32
global start
extern exit
import exit msvcrt.dll
segment data use32 class=data:
    a dw 1,2,3,4
    l equ ($-a)/2 → dw
    sum dw 0
segment code use32 class=code:
  start:
      xor bx,bx
      mov ecx,l
      mov esi,0
      jecxz end_loop
      start_loop:
          mov ax, [a+esi]
          add bx, ax
          add esi,2 → dw
      loop start_loop
      end_loop:
      mov [sum], bx
      push dword 0
      call [exit]
```

$ → current address

$$ → start of address.

```
0100 0200 0300 0400 ← DS
↑    ↑    ↑    ↑
0    2    4    6
```

↓

AX = 0002

2. Given s - array of dwords, obtain D of D(i) = S(i) % S(i+1), signed.

s: 10,3,5,2 → d: 1,3,1.

```
segment data use32 class=data:
    s dd 10,3,5,2
    l equ ($-s)/4 → dd
    d times(l-1) dd 0 → create table de (l-1) elemente
segment code use32 class=code:
  start:
      mov ecx, l-1 ; ecx ← 2
      mov edi, 0 → index for d
      mov esi, 0 → index for s
```

```
jecxz end_loop:
start_loop: → parcurgem d
    mov eax, [s+esi]
    cdq
    idiv dword [s+esi+4] → s[i] / s[i+1]
    mov [d+edi], edx → %
    add edi, 4
    add esi, 4
loop start_loop
end_loop:
```

(=>)

```
mov eax, [s+4*esi]
cdq
idiv dword [s+4*(esi+1)]
mov [d+4*edi], edx
inc esi
inc edi
```

```
s+4·(esi+1)
inc esi
inc edi
```

3. Given s-string of bytes, copy in d multiples of 7 from s in reverse order, unsigned.

```
segment data use32 class=data:
    s db 20,14,70,44,21
    l equ ($-s)
    d times l db 0

segment code use32 class=code:
  start:
    mov esi, l-1    „←"
    mov edi, 0      „→"
    mov ecx, l
    jecxz end_loop:
    start_loop:
      mov al, [s+esi]   } ax ← elem. from s.
      mov ah, 0         }
      mov bl, 7
      div bl  ← ax : bl = al, ah
      cmp ah, 0
      jne skip  ← if the remainder is ≠ 0, we don't add it to d.
          mov al, [s+esi] ← reload the original value
          mov [d+edi], al ← write to d
          inc edi
      skip:
      dec esi
    loop start_loop
    end_loop:
```

Sha-boop!